

Technical Test Candidate Guide

You are contracting for **Warung Books**, a seed-stage bookkeeping app whose non-technical founder built it with an AI app builder in a weekend. It now has 1,400 paying customers, an accountant about to review the books, and a bank partnership that hinges on a security review. This repository is their codebase. Fix the planted defects, remove the UI slop, move it onto a real database, harden it for production, and deploy it — with a logged AI prompt trail and your own sub-agents. Then write the infrastructure plan the bank needs and the roadmap the investors want.

CLIENT SCENARIO

Warung Books — micro-business bookkeeping

DOCUMENT

Candidate guide

SCAFFOLD VERSION

0.1.0

SUGGESTED TIME

6–8 hours

STAGE

Seed · 1,400 customers · audit & bank review pending

REPOSITORY

technical-test-microservice

GENERATED

2026-09-23

STACK

TypeScript · Node · Hono · React · PostgreSQL

Start here. Run `pnpm install`, then `pnpm test` (green) and `pnpm --filter @ledgerlab/ledger-api test:challenges` (red on purpose). The red suite is your first task.

Contents

1. The client and the mandate

Warung Books: how a vibe-coded app got 1,400 customers, why it must go production, and what "good" looks like.

`docs/CLIENT-STORY.md`

2. Overview, goals and scoring

What the scaffold is, goals G0–G8, the rubric and the submission checklist.

`README.md`

3. Task backlog

Ordered P0–P3 work: fix the defects, then quality, database, deploy and security.

`docs/TASKS.md`

4. Design rules (anti-slop)

The enforced UI rules: colour balance, icons, typography, surfaces, motion.

`docs/DESIGN.md`

5. Database

The storage port, PostgreSQL wiring, and MySQL/SQLite/SQL Server options.

`docs/DATABASE.md`

6. Deployment

Render/AWS/GCP/Azure runbooks and the definition of production-scale.

`docs/DEPLOYMENT.md`

7. Security

Secrets, the service-to-service boundary, CORS, headers and rate limiting.

`docs/SECURITY.md`

8. AI usage policy

How to log every AI prompt and what is scored; includes rejected prompts.

`docs/AI-USAGE.md`

9. Sub-agents

Build at least three sub-agents and one of your own; scope and contracts.

`docs/SUBAGENTS.md`

10. Infrastructure plan (template)

The scored deliverable: topology, durability, scaling, cost, DR and ADRs.

`docs/INFRASTRUCTURE-PLAN.md`

11. Next-phase development plan (template)

The scored deliverable: outcomes, prioritisation, milestones and capacity.

12. Submission template

The fill-in-the-blanks document a reviewer reads first.

docs/SUBMISSION.md

1. The client and the mandate

Scenario brief. This is fiction, but work it as a real engagement. Every technical goal in this test exists because of something on this page.

The client

Company	Warung Books (PT Warung Buku Digital), Jakarta
Product	Bookkeeping and cash-flow tracking for Indonesian micro-businesses
Founder & CEO	Kira Mandala — ex-café owner, not an engineer
Stage	Seed. 1,400 paying businesses, growing ~30% month over month
Throughput	~62,000 journal lines and Rp 4.1bn of recorded value per month
Handing over	The codebase extracted into this repository
You are	The first senior engineer they have hired

How it got here

Kira ran a small café and could not afford an accountant. One weekend she built the first version of Warung Books **with an AI app builder** — describing screens in plain language, pasting whatever the model produced, shipping it. It worked well enough that she shared it in a few WhatsApp groups for café owners. It spread. Within a year, 1,400 businesses depended on it to know whether they were making money.

The growth was real. The engineering underneath it was not:

- The whole app is **one service** holding the database, the ledger, and the reporting logic. It falls over when the monthly reconciliation job runs.
- It uses a **SQLite file on a single VM**. There are no backups anyone has tested. If that VM dies, so does the company.
- CORS is `*`. There is **one shared API password in the repo**, and it has been rotated exactly never.
- There are **no automated tests**. Kira fixes things by clicking around the UI.
- Nobody has ever checked whether the ledger's debits equal its credits. It is an accounting product; that is the one thing that must be true.

Some of the mess has already been cleaned up in the extraction: money is stored as integer minor units, the ledger and reporting concerns are separated, and a storage port exists so the database can be swapped. Two real defects were left in **on purpose** — they are the ones that scare Kira's accountant.

What you are inheriting

SYMPTOM IN THE STORY	EVIDENCE IN THIS REPO	RISK
"Reports for a month bleed into the next"	<code>buildTrialBalance</code> ignores <code>asOf</code> (Challenge A)	Misstated financials; audit failure
"Entries get voided twice and the trail looks wrong"	<code>voidJournalEntry</code> has no state guard (Challenge B)	Audit-trail defect; looks like fraud
"It's a website, not an application"	<code>packages/ui</code> is usable but inconsistent	Customers don't trust money UIs that look like demos
"One VM, one file"	<code>InMemoryLedgerRepository</code> is the default	Total data loss on restart/host failure
"The repo has a password in it"	see <code>.env.example</code> and <code>docs/SECURITY.md</code>	Credential compromise
"Anyone can call the internal API"	<code>/api/internal/*</code> is public unless the token is set	Data exfiltration
"It fell over at 9pm on payday"	Single instance, no autoscaling, no health checks	Revenue and reputation loss
"We can't answer the bank's security questionnaire"	No WAF, no rate limiting, no real domain, no headers	The partnership dies

Why now (the burning platform)

1. **An audit that matters.** Kira's accounting partner, a licensed CPA, has agreed to review Warung Books' output — but only if the trial balance is correct and the void trail is defensible. Review starts **mid-October**.
2. **A bank partnership.** A working-capital lender wants to embed Warung Books so their micro-business clients can qualify for loans. Their vendor security review requires: a managed database with tested backups, no secrets in source control, rate limiting, a WAF, HTTPS on a **domain they recognise** (not a `*.onrender.com` URL), and evidence of uptime monitoring. Targeting **17 October 2026**.
3. **A seed round.** Due diligence opens **28 November 2026**. Investors will ask for uptime numbers and a data-loss story.
4. **Growth.** Traffic tripled last quarter. The single instance is already hitting CPU limits during the nightly reconciliation.

The mandate

Take Warung Books from a vibe-coded prototype to something a bank and an accountant will trust — without breaking the 1,400 businesses already using it.

That is exactly this repository's goals:

MANDATE	GOAL
Fix the defects the accountant will find	G0, G1
Make the money UI trustworthy	G2
Move off a single in-memory/file store to a real, backed-up database	G3
Survive payday traffic; deploy reproducibly	G4
Pass the bank's security questionnaire	G5
A domain and edge the bank recognises	G6
Work in a way a future team can audit	G7, G8
Give the bank a plan they can approve, with tested backups and a costed path to 10×	G9
Give investors a credible plan for the phase after this one	G10

What "good" looks like

MEASURE	TODAY	TARGET
Trial balance correctness	Unverified	Correct for any asOf ; regression-tested
Void transition	Unguarded	POSTED → VOID only; 409 otherwise
Availability	Best effort	99.9% with health-checked replicas
p95 API latency	Unknown	< 300 ms on the ledger reads
Backups	None tested	Daily, with a tested restore
Secrets in git	Yes	Zero; all from a secret store
Public internal API	Yes	Blocked at the edge
Domain	Provider subdomain	ledgerlab.example.com on a real TLD, behind Cloudflare

Stakeholders and what they care about

WHO	CARES ABOUT	SOUNDS LIKE
Kira (CEO)	Not losing customers; hitting the bank date	"I don't need it pretty. I need it to not lose data."
CPA reviewer	The numbers tie out	"If debits don't equal credits, I can't sign off."
Bank security reviewer	Evidence, not intent	"Show me the WAF rule and the /api/internal verdict."
Ops (Budi)	Monthly close not breaking	"The 9pm reconciliation is where it dies."
Existing customers	The app keeps working	"My books from last year — are they still there?"
Future engineers	Not inheriting another mess	"Why did this change? Where's the prompt log?"

Constraints

- **Two weeks, one engineer.** Underscope deliberately and document what you skipped.
- **Budget is real.** Free/low tiers are fine; say what you chose and why.
- **Do not break existing customers.** Migrate the data; keep the API contract.
- **Money integrity is non-negotiable.** Never floats for money; never an unbalanced posted entry; the audit trail is append-only.
- **No downtime window longer than five minutes.** Roll forward, not sideways.
- **The demo data is not real.** Never paste customer data into an AI tool.

Risks

RISK	MITIGATION
A quick fix breaks an accounting invariant	The challenge suite + port-level tests gate every change
Migration loses or double-counts entries	Idempotent seed/migrate; verify totals before and after
Security work left until the end and rushed	Start <code>INTERNAL_API_TOKEN</code> + CORS on day one
Bank review needs evidence you did not capture	Keep commands, <code>curl -I</code> output, WAF screenshots as you go
Two weeks isn't enough	Ship G0–G5 fully; treat Cloudflare/TLD as the stretch

The one-sentence brief

Make the ledger provably correct, the data durable, the service survive payday, and the whole thing defensible in a security review — then prove it with tests, evidence, and a prompt log.

2. Overview, goals and scoring

A **double-entry accounting** scaffold: a React dashboard, a **ledger API**, a **reporting API**, and shared domain packages in a pnpm/Turborepo monorepo.

Your job is **not** to build a finished product. It is to take this scaffold, **fix its intentional defects**, **remove the UI slop**, **harden it for production**, and **deploy it** — while logging every AI prompt you use and building your own sub-agents to do the work.

Read this file end to end before writing code. Then read `docs/TASKS.md` for the ordered backlog.

The scenario

You have been hired by **Warung Books**, a seed-stage bookkeeping app for Indonesian micro-businesses. Its non-technical founder built it with an AI app builder in a weekend; it now has 1,400 paying customers, an **accountant about to review the numbers**, a **bank partnership with a security review**, and a seed round in due diligence. This repository is their codebase. Every technical goal below exists because of something in that story.

Read the full brief first: `docs/CLIENT-STORY.md`.

1. What already works

AREA	STATUS
Monorepo (pnpm workspaces + Turborepo)	✅ builds, typechecks, tests
Double-entry domain (integer minor units, signed amounts)	✅ with unit tests
Ledger API: accounts, journal entries, trial balance, void	✅ tested
Reporting API: dashboard, income statement, balance sheet	✅ tested
Postgres adapter (Drizzle) + in-memory adapter	✅ same port
Dashboard: dashboard, ledger, new entry, accounts, reports	⚠️ functional, needs review
Dockerfiles + Render blueprint + AWS/GCP/Azure/Cloudflare notes	✅ present
Challenge suite (2 failing specs that define required fixes)	❌ red on purpose

Run the default suite and everything is green. Run the challenge suite and you will see exactly what is broken:

```
pnpm install
pnpm test # green
pnpm --filter @ledgerlab/ledger-api test:challenges # red — this is the work
```

2. Goals

Each goal has acceptance criteria. "Done" means the criteria are met **and evidenced** (a command, a file, or a screenshot in your submission).

G0 — Fix the errors (highest priority)

- ☐ Make `pnpm --filter @ledgerlab/ledger-api test:challenges` green by fixing the **source**, not the tests. The two defects are documented in `apps/ledger-api/src/challenges/`.
- ☐ `pnpm typecheck && pnpm test && pnpm build` all pass on a clean checkout.
- ☐ No `TODD0(candidate)` left unresolved in code paths you claim are finished.

G1 — Finish the core ledger

- ☐ Every mutation path preserves the invariant: **sum of signed minor units === 0** for each entry.
- ☐ Voiding is a guarded `POSTED → VOID` transition (no re-voiding).
- ☐ Trial balance, income statement, and balance sheet are correct for any `as0f` / date range.
- ☐ Tests cover each rule; each new test would fail if the rule broke.

G2 — Un-slop the UI

- ☐ Dashboard reads as a product, not a template: clear hierarchy, one accent colour, no decorative icon boxes, no emojis, no gradients/glassmorphism.
- ☐ Money is right-aligned and uses tabular figures; empty and error states are designed, not default.
- ☐ Follow `docs/DESIGN.md` (the enforced design rules).

G3 — Real database (required)

- ☐ The ledger runs against a **real database**, not the in-memory adapter. PostgreSQL is wired up; **MySQL, MariaDB, SQLite/libSQL, or SQL Server are acceptable** if you implement the `LedgerRepository` port.
- ☐ Schema is migrated (not auto-synced) and seeded via a repeatable script.
- ☐ See `docs/DATABASE.md`.

G4 — Production-scale deployment

- ☐ Deployed to **at least one** of Render / AWS / GCP / Azure, reachable over HTTPS, with `/health` monitored.
- ☐ Stateless services, ≥ 2 replicas (or `min-instances ≥ 1`), managed database, connection pooling, graceful shutdown.
- ☐ A **one-command** reproducible deploy is documented and was actually run.
- ☐ See `docs/DEPLOYMENT.md`.

G5 — Security-aware hardening

- ☐ No secrets in git; all from the platform secret store.

- ☐ `INTERNAL_API_TOKEN` set and `/api/internal/*` unreachable from the public internet.
- ☐ `CORS_ORIGINS` restricted to the real dashboard origin (never `*` in prod).
- ☐ HTTPS/HSTS, secure headers, least-privilege DB user, no stack traces in API error bodies, rate limiting in front of `/api/*`.
- ☐ See `docs/SECURITY.md`.

G6 — Cloudflare + custom TLD domain (scored bonus)

- ☐ A real TLD you control (not `*.onrender.com` / `*.run.app`).
- ☐ Cloudflare proxying, TLS Full (strict), WAF managed rules, and a rate-limit rule on `/api/*`.
- ☐ `/api/internal/*` blocked at the edge; cache rules separate static assets from API traffic.
- ☐ Evidence: `dig`, `curl -I`, and a WAF event screenshot.
- ☐ See `deployment/cloudflare/README.md`.

G7 — AI usage with a prompt log (required)

- ☐ Every AI prompt that influenced the code is recorded via `pnpm ai:log`.
- ☐ `pnpm ai:verify` passes in CI.
- ☐ Each entry has: tool, model, task, prompt, summary, decision, files.
- ☐ Show your rejected AI suggestions too — judgment is what is scored.
- ☐ See `docs/AI-USAGE.md`.

G8 — Build your own sub-agents (required)

- ☐ At least three working sub-agents under `.opencode/agent/`.
- ☐ Starter agents are provided (`ledger-architect`, `ui-unslop`, `test-runner`, `deploy-security`) — you must extend or replace them with agents that fit your workflow, and demonstrate them working.
- ☐ See `docs/SUBAGENTS.md`.

G9 — Infrastructure plan (required)

- ☐ Produce `docs/INFRASTRUCTURE-PLAN.md` from the template: target topology, environments, compute and scaling, data durability with RPO/RTO and an actually-performed restore drill, networking/edge, secrets and identity, CI/CD and rollback, observability with SLOs, a cost model with arithmetic, failure modes, and at least three ADRs with rejected options.
- ☐ Every section is specific and matches what is really deployed.
- ☐ See `docs/INFRASTRUCTURE-PLAN.md`.

G10 — Next-phase development plan (required)

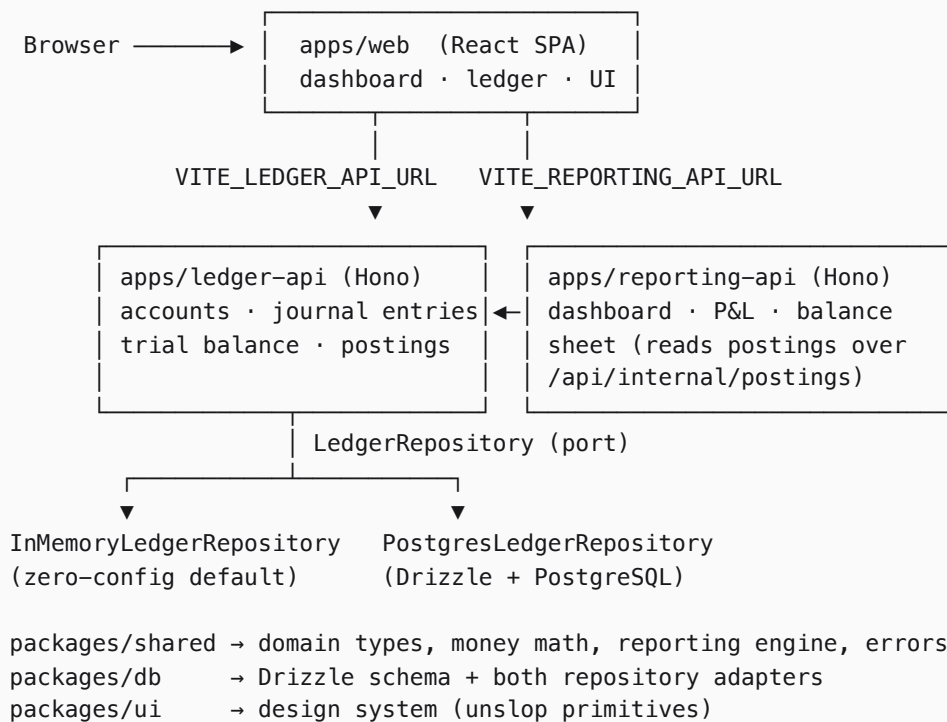
- ☐ Produce `docs/NEXT-PHASE-PLAN.md`: measurable outcomes tied to stakeholders, a stated prioritisation method, three milestones with testable exit criteria and dates, delivery capacity and the next hires, technical workstreams, risks, metrics with sources, and what you are explicitly deferring.
- ☐ See `docs/NEXT-PHASE-PLAN.md`.

3. Scoring rubric

#	CRITERION	WEIGHT	WHAT EARNS FULL MARKS
1	Correctness — G0/G1	20	Challenge suite green by fixing source; invariant proven by tests; no regressions
2	Engineering quality — G1/G3	12	Clean ports/adapters, migrations, typed boundaries, readable tests
3	UI quality — G2	12	Design rules met; dashboard is clear and trustworthy
4	Production-scale deploy — G4	12	Reproducible deploy, health checks, scaling, managed DB, pooling
5	Security — G5	12	Secrets, CORS, internal-token boundary, headers, least privilege
6	Infrastructure plan — G9	12	Specific, deployable-from-the-doc, tested restore, costed, ADR-backed
7	Next-phase plan — G10	10	Outcomes, prioritisation method, milestones with exit criteria, capacity honesty
8	AI prompt log — G7	5	Complete, honest, includes decisions and rejected suggestions
9	Sub-agents — G8	5	≥3 agents, clearly scoped, demonstrably used
10	Cloudflare + TLD — G6 (bonus)	+5	Real domain, proxied, Full (strict), WAF + rate limit, evidence
11	Extra credit	+5	Multi-currency reporting, audit trail, CSV/PDF export, OpenAPI, load test

Automatic fail: tests edited to pass without fixing source; secrets committed; `CORS_ORIGINS=*` in production; fake AI log entries; a plan document submitted with `TODO` s still in it.

4. Architecture



Key invariant. `amountMinor` is a **signed integer**: > 0 is a debit, < 0 is a credit. For every entry, $\Sigma \text{amountMinor} === 0$. Never introduce floating point for money.

5. Repository map

apps/	
web/	React + Vite dashboard (Tailwind v4)
ledger-api/	Accounts, journal entries, trial balance
src/app.ts	Hono app factory (testable, no listen)
src/services/	Business rules (LedgerService)
src/repositories/	Storage adapter selection
src/challenges/	🔴 Red specs that define your work
reporting-api/	Reports built from postings (service-to-service)
packages/	
shared/	Domain types, money, reporting engine, seed, errors
db/	Drizzle schema, Postgres + in-memory adapters, seed
ui/	Design system (Button, Card, Stat, Table, ...)
deployment/	
Dockerfile.*, nginx.conf, render.yaml	
aws/ gcp/ azure/ cloudflare/	
docs/	Scenario, tasks, design, database, deployment, security, AI, sub-agents, infrastructure plan, next-phase plan, submission
scripts/	log-ai-prompt.mjs, verify-ai-log.mjs
.opencode/agent/	Your sub-agents live here

6. Quick start

Requirements: **Node ≥ 20.11**, **pnpm 10** (`corepack enable`). Docker optional.

```
pnpm install

# Terminal 1 – ledger API          (in-memory, seeded, no DB needed)
pnpm --filter @ledgerlab/ledger-api dev

# Terminal 2 – reporting API
pnpm --filter @ledgerlab/reporting-api dev

# Terminal 3 – dashboard → http://localhost:5173
pnpm --filter @ledgerlab/web dev
```

Or run everything: `pnpm dev` (Turborepo, parallel).

Useful commands:

```
pnpm typecheck          # all packages
pnpm test                # green baseline
pnpm build               # bundles APIs + builds the SPA
pnpm ai:log -- --help    # log an AI interaction
pnpm ai:verify           # CI gate for the prompt log
pnpm --filter @ledgerlab/ledger-api test:challenges # red – your work
```

Copy `.env.example` to `.env` if you want to override ports or point at a database.

7. Databases

The scaffold ships two adapters behind one port. The **default is in-memory** so reviewers can run it instantly; **G3 requires you to use a real database**.

OPTION	NOTES
PostgreSQL	Wired via Drizzle: <code>packages/db/src/postgres-repository.ts</code> . Recommended.
MySQL / MariaDB	Implement <code>LedgerRepository</code> and add a Drizzle MySQL schema.
SQLite / libSQL	Great for local + edge; same port.
SQL Server	Acceptable; add the Drizzle/mssql adapter.

```
cp .env.example .env
# DATABASE_URL=postgres://ledgerlab:ledgerlab@localhost:5432/ledgerlab
pnpm --filter @ledgerlab/db migrate:sql
pnpm --filter @ledgerlab/db seed
```

Details, schema, and the port contract: `docs/DATABASE.md`.

8. Deployment

Pick one target and make it reproducible. Notes for all of:

- **Render** — `deployment/render.yaml` Blueprint (DB + 2 services + static site).
- **AWS** — `deployment/aws/` (ECS Fargate, RDS, S3+CloudFront).
- **GCP** — `deployment/gcp/` (Cloud Run, Cloud SQL).
- **Azure** — `deployment/azure/` (Container Apps, Flexible Server).
- **Cloudflare + TLD** — `deployment/cloudflare/` (scored bonus).

Full walkthrough and the definition of "production-scale": `docs/DEPLOYMENT.md`.

9. AI usage rules

AI is allowed and expected. The rule is simple: **if an AI helped, it is logged.**

```
pnpm ai:log -- \
  --tool "opencode" --model "deepseek/deepseek-v4.1-flash" \
  --subagent "ledger-architect" \
  --task "Fix trial balance asOf filtering" \
  --prompt "buildTrialBalance ignores asOf; make it filter postings by date" \
  --summary "Added date filter in reporting aggregate; kept balanced flag" \
  --decision "accepted" \
  --files "packages/shared/src/reporting.ts"
```

This appends to `docs/ai/prompt-log.jsonl` and regenerates `docs/AI-PROMPT-LOG.md`. `pnpm ai:verify` (run in CI) fails on missing or incomplete records. Full policy: `docs/AI-USAGE.md`.

10. Sub-agents

You must build sub-agents and use them for the work. Starter agents exist under `.opencode/agent/`; extend or replace them. Suggested additions:

AGENT	JOB
<code>ledger-architect</code> <i>(provided)</i>	Owns the double-entry invariant and posting rules
<code>ui-unslop</code> <i>(provided)</i>	Enforces the design rules on every screen
<code>test-runner</code> <i>(provided)</i>	Runs typecheck/test/build and reports minimal fixes
<code>deploy-security</code> <i>(provided)</i>	Production-scale + security review of the deploy
<code>db-migrator</code> <i>(suggested)</i>	Schema, migrations, seeding, and the repository port
<code>reporting-verifier</code> <i>(suggested)</i>	Proves reports tie out to the ledger

Spec and definition of done: `docs/SUBAGENTS.md`.

11. Submission

1. Push a branch; keep the build green (`pnpm typecheck && pnpm test && pnpm build`).
2. Attach **deployed URLs** (dashboard + APIs) and `/health` responses.
3. Include the **AI prompt log** (`docs/AI-PROMPT-LOG.md`) and your **sub-agent** definitions (`.opencode/agent/`).
4. Fill in both planning deliverables: `docs/INFRASTRUCTURE-PLAN.md` (G9) and `docs/NEXT-PHASE-PLAN.md` (G10).
5. Add `docs/SUBMISSION.md` with: what you changed, what you skipped and why, your deploy commands, Cloudflare/TLD evidence, and known limitations.

Suggested time budget: 6–8 hours. G0 + G1 first (correctness), then G2, then G3–G5, then G6 if time allows. Underscope deliberately and document it.

12. Non-goals

- Real auth / multi-tenancy (mention how you would add it; do not build it).
- Payments, invoicing, payroll, tax filing.
- A pixel-perfect design system — clarity beats polish.
- 100% test coverage — test the invariants and the edges that matter.

3. Task backlog

Context: `CLIENT-STORY.md`. You are taking **Warung Books** from a vibe-coded prototype to production. The backlog below is the plan.

Ordered backlog. Work top to bottom; stop and document when you run out of time. Every completed item should have evidence (a test, a command, or a screenshot).

Legend: **P0** blocks everything · **P1** core · **P2** required for full marks · **P3** stretch.

P0 — Fix the errors

- ☐ `pnpm --filter @ledgerlab/ledger-api test:challenges` is red. Make it green **by fixing production code**, never the specs.
 - **Challenge A** (`src/challenges/asof.challenge.ts`) — `buildTrialBalance` ignores `asOf`, so periods bleed into each other. Fix the date filtering in `packages/shared/src/reporting.ts`.
 - **Challenge B** (`src/challenges/void.challenge.ts`) — voiding is an unguarded state change. Make `POSTED → VOID` the only legal transition and return `409 CONFLICT` otherwise. Touch `LedgerService` and both repository adapters.
- ☐ `pnpm typecheck && pnpm test && pnpm build` all pass from a clean clone.

P1 — Core ledger correctness

- ☐ Add a test that proves the balancing invariant for the Postgres adapter too (or for the port in general), not just the in-memory one.
- ☐ Reject journal lines that reference an **inactive** account.
- ☐ Reject entries dated in a closed accounting period (add a `periods` table or a simple `closedThrough` setting; document the rule you chose).
- ☐ Income statement and balance sheet must exclude `VOID` entries — add a regression test.
- ☐ `pnpm --filter @ledgerlab/ledger-api test:challenges` green in CI.

P2 — Product quality

- ☐ Un-slop the dashboard per `DESIGN.md`: remove decorative icons, gradients, excess nested cards; fix hierarchy and spacing.
- ☐ Empty, loading, and error states on every page.
- ☐ Keyboard access: the ledger list is navigable, the entry form is usable without a mouse.
- ☐ Money input rejects bad values with an inline message (uses `parseAmountToMinor` errors).
- ☐ Pagination and filtering on the ledger list (status, date range).

P2 — Database

- ☐ Run against a real database (`docs/DATABASE.md`). PostgreSQL is wired; MySQL/SQLite are accepted if you implement the port.
- ☐ Migrations applied as a deploy step, not on boot.
- ☐ Seeding is idempotent and scripted.
- ☐ Connection pooling configured and documented.

P2 — Deploy, scale, secure

- ☐ Deploy to Render / AWS / GCP / Azure (one is enough).
- ☐ Health checks wired; ≥ 2 replicas or `min-instances ≥ 1`.
- ☐ Secrets in the platform store; **no** secret in the repo.
- ☐ `CORS_ORIGINS` restricted; `INTERNAL_API_TOKEN` set.
- ☐ HSTS + secure headers at the edge.
- ☐ Rate limit on `/api/*`.
- ☐ Migrations run as a job/step, not at container start.

P2 — Infrastructure plan (G9)

- ☐ Copy `INFRASTRUCTURE-PLAN.md` to `docs/INFRASTRUCTURE-PLAN.md` and fill every section.
- ☐ Draw the target topology and make sure it matches what is actually deployed.
- ☐ State RPO/RTO and **perform a restore drill**; record the duration and result.
- ☐ Show the capacity arithmetic (requests/sec per replica, DB connections, storage growth).
- ☐ Cost the plan at $1\times$ / $3\times$ / $10\times$ traffic and name what breaks first.
- ☐ Write ≥ 3 ADRs with genuinely rejected options.
- ☐ Verify the security controls table links to evidence, not intent.

P2 — Next-phase development plan (G10)

- ☐ Copy `NEXT-PHASE-PLAN.md` to `docs/NEXT-PHASE-PLAN.md` and fill every section.
- ☐ Define 3–5 measurable outcomes tied to a named stakeholder.
- ☐ Prioritise with a stated method (RICE or your own) and show the trade-offs.
- ☐ Three milestones with testable exit criteria, working back from bank go-live.
- ☐ Be honest about capacity; name the next two hires and why.
- ☐ List what you are explicitly deferring, with reasons.

P3 — Bonus (Cloudflare + TLD)

- ☐ Custom domain on a real TLD, proxied through Cloudflare.
- ☐ TLS Full (strict), Always Use HTTPS, HSTS.
- ☐ WAF managed rules + a rate-limit rule.
- ☐ `/api/internal/*` blocked at the edge.

- ☐ Cache rules: assets cached, API bypassed.
- ☐ Evidence committed (`dig` , `curl -I` , WAF event screenshot).

P3 — Extra credit

- ☐ Multi-currency: reports must not sum across currencies silently. Group by currency or require an explicit FX rate.
 - ☐ Audit trail: who changed what, when.
 - ☐ CSV or PDF export for the trial balance.
 - ☐ OpenAPI spec generated from the route schemas.
 - ☐ A load test (`k6` / `autocannon`) with results committed.
-

Definition of done (whole test)

1. `pnpm typecheck && pnpm test && pnpm build` green.
2. `pnpm --filter @ledgerlab/ledger-api test:challenges` green.
3. `pnpm ai:verify` green and the log is honest.
4. Deployed URL with `/health` returning `200` .
5. `docs/INFRASTRUCTURE-PLAN.md` filled in, with a performed restore drill.
6. `docs/NEXT-PHASE-PLAN.md` filled in, with milestones and exit criteria.
7. `docs/SUBMISSION.md` written: changes, decisions, evidence, limitations.

4. Design rules (anti-slop)

The dashboard must look like a product, not a template. These rules are enforced; the `ui-unslop` sub-agent reviews against them.

Colour — 70 / 20 / 10

- 70% neutral base: `zinc` backgrounds, white surfaces, `zinc-900` text.
- 20% supporting: borders, muted text, hover states.
- 10% accent: **one** accent colour (`indigo-600`) for primary actions and focus. Semantic colours only for state: `emerald` (positive/balanced), `amber` (warning), `red` (negative/error).
- Do **not** give each card its own tint. Let whitespace separate things.

Icons

- Icons are for **actions and navigation only**, from **one** library (`lucide-react`), rendered inline.
- **Never** put an icon in a coloured rounded square/circle.
- If removing an icon loses no information, remove it.

Typography

- Sans-serif only (`Inter` , system UI). **No decorative serif headlines.**
- Establish hierarchy with size/weight/colour, not with boxes.
- Money and quantities use `tabular-nums` and are **right-aligned**.

Surfaces

- Solid backgrounds. **No glassmorphism / `backdrop-filter`** .
- **No gradients** on text, buttons, or cards. **No coloured/glowing shadows.**
- Shadows are subtle, neutral, and indicate elevation only (`shadow-xs`).
- **No cards nested in cards.** If you need grouping, use a heading and spacing.
- Never combine `border-left` with `border-radius` on a container.

Motion

- No entrance animations on content. Transitions only for state feedback (hover, focus, loading), 150–300 ms.
- Never animate content in a way that breaks when it is already in the viewport.

Empty, loading, error states

- Every async surface has all three, designed — not a spinner in the corner.

- Errors are human sentences, not stack traces.

Do not ship

- Emojis as visual assets.
- A floating chat bubble.
- More than one accent hue competing for attention.
- Placeholder "Lorem ipsum" or fake statistics.

Checklist before you claim G2 done

- ☐ Squint test: the page is not one blob of similar colour.
- ☐ Exactly one accent colour in use.
- ☐ No icon-in-rounded-square anywhere.
- ☐ No gradient, no glass, no coloured shadow.
- ☐ No card-in-card.
- ☐ Money columns are tabular and right-aligned.
- ☐ Keyboard focus is visible on every interactive element.
- ☐ Loading / empty / error states exist on every data surface.

5. Database

The scaffold defines one storage port and ships two adapters. **G3 requires you to run against a real database.** PostgreSQL is wired end to end; **MySQL, MariaDB, SQLite/libSQL, and SQL Server are all acceptable** if you implement the port.

The port

```
// packages/shared/src/repository.ts
export interface LedgerRepository {
  readonly kind: "memory" | "postgres"; // extend the union for new engines
  listAccounts(): Promise<Account[]>;
  getAccountById(id: string): Promise<Account | undefined>;
  getAccountByCode(code: string): Promise<Account | undefined>;
  createAccount(input: CreateAccountInput): Promise<Account>;

  listJournalEntries(query: ListJournalEntriesQuery): Promise<Paginated<JournalEntry>>;
  getJournalEntry(id: string): Promise<JournalEntry | undefined>;
  createJournalEntry(input: CreateJournalEntryInput): Promise<JournalEntry>;
  voidJournalEntry(id: string): Promise<JournalEntry | undefined>;

  /** POSTED lines joined with account metadata – the source for all reports. */
  listPostings(): Promise<PostingRow[]>;
}
```

Business rules live in `LedgerService`, **not** in the adapter. An adapter only persists and reads back; it must not silently "fix" an unbalanced entry.

Option A — PostgreSQL (wired, recommended)

- Schema: `packages/db/src/schema.ts` (Drizzle, `pg-core`).
- Adapter: `packages/db/src/postgres-repository.ts`.
- Migration SQL: `packages/db/migrations/0000_init.sql`.

```
cp .env.example .env
# DATABASE_URL=postgres://ledgerlab:ledgerlab@localhost:5432/ledgerlab

# Apply the schema (idempotent SQL). Or use drizzle-kit generate/migrate
# once you start evolving the schema.
pnpm --filter @ledgerlab/db migrate:sql

# Load the demo chart of accounts + journal entries (idempotent)
pnpm --filter @ledgerlab/db seed
```

Selecting the adapter is automatic: if `DATABASE_URL` is set, `resolveLedgerRepository()` uses Postgres; otherwise it falls back to the seeded in-memory adapter.

Local Postgres without Docker:

```
createuser ledgerlab --pwprompt
createdb ledgerlab -O ledgerlab
```

Or with Docker (if available):

```
docker run --name ledgerlab-db -e POSTGRES_USER=ledgerlab -e POSTGRES_PASSWORD=ledgerlab \
-e POSTGRES_DB=ledgerlab -p 5432:5432 -d postgres:16
```

Option B — MySQL / MariaDB

1. Add `mysql2` and a `mysqlTable` schema in `packages/db/src/schema.mysql.ts`.
2. Implement `MySqlLedgerRepository` against the same port.
3. Generate migrations with `drizzle-kit` (`dialect: "mysql"`).
4. Extend `resolveLedgerRepository()` to pick it from `DATABASE_URL` (e.g. the `mysql://` scheme) and widen the `kind` union.

Option C — SQLite / libSQL

Useful for local dev and edge. Implement the port over `better-sqlite3` or `@libsql/client`, store money as `INTEGER` (never `REAL`), and enable `PRAGMA foreign_keys = ON` plus WAL mode.

Option D — SQL Server

Implement the port with `mssql/Drizzle` `mssql-core`. Use `BIGINT` for `amount_minor` and a `VARCHAR(4)` unique index on `accounts.code`.

Data model

```
accounts(id, code UNIQUE, name, type, currency, is_active, created_at)
journal_entries(id, entry_date, memo, reference, status, created_at)
journal_lines(id, entry_id → journal_entries, account_id → accounts,
              amount_minor, position, memo)
```

Money is `amount_minor`, a signed integer: `> 0` debit, `< 0` credit. No `REAL` / `FLOAT` / `DOUBLE` column may ever hold money.

Migrations

- Never auto-sync schema in production. Run migrations as a deploy step or a one-off job, with the DB user that owns the schema.
- The app's runtime user should be least-privilege (`SELECT/INSERT/UPDATE` on the three tables; no DDL outside the migration step).
- Keep migrations forward-only and idempotent where possible (`CREATE TABLE IF NOT EXISTS`).

Seeding

`packages/shared/src/seed.ts` defines the demo chart of accounts and entries. `seed()` on the in-memory adapter and `pnpm --filter @ledgerlab/db seed` against Postgres both consume it, so behaviour matches. Seeding must be safe to re-run.

Production checklist

- ☐ Managed database (RDS / Cloud SQL / Azure Flexible Server / Render Postgres) in a private network; not publicly reachable.
- ☐ TLS required; verify the certificate.
- ☐ Connection pool sized to the plan (start at 10); enable an idle timeout.
- ☐ Automated backups with a retention policy; **test a restore**.
- ☐ Point-in-time recovery or daily snapshots.
- ☐ Alerts on connection saturation and slow queries.
- ☐ Migrations applied before the new version starts serving traffic.

6. Deployment

Deploy to **at least one** of Render / AWS / GCP / Azure, over HTTPS, with health checks and a managed database. Then, for bonus marks, put **Cloudflare** in front on a **real TLD**.

Per-cloud runbooks:

- Render — `deployment/render.yaml`
- AWS — `deployment/aws/README.md`
- GCP — `deployment/gcp/README.md`
- Azure — `deployment/azure/README.md`
- Cloudflare + TLD — `deployment/cloudflare/README.md`

Recommended path: Render (fastest to a real URL)

```
# 1. Push the repo to GitHub.
# 2. Render Dashboard → New → Blueprint → select the repo.
#    The blueprint in deployment/render.yaml provisions:
#      - ledgerlab-db          (managed PostgreSQL)
#      - ledgerlab-ledger-api   (Docker, public, /health)
#      - ledgerlab-reporting-api (Docker, public, /health)
#      - ledgerlab-web         (static SPA)
# 3. Fill the `sync: false` values: CORS_ORIGINS and the two VITE_* URLs.
# 4. Run migrations once (Render Shell or a one-off job):
#     pnpm --filter @ledgerlab/db migrate:sql
#     pnpm --filter @ledgerlab/db seed
```

`INTERNAL_API_TOKEN` is generated by Render and shared to the reporting service via `fromService`, so `/api/internal/*` stays service-to-service only.

Environment variables

VARIABLE	SERVICE	REQUIRED	NOTES
<code>DATABASE_URL</code>	ledger-api	prod	Postgres/MySQL connection string
<code>LEDGER_API_PORT</code>	ledger-api	no	default <code>4001</code>
<code>REPORTING_API_PORT</code>	reporting-api	no	default <code>4002</code>
<code>LEDGER_API_URL</code>	reporting-api	yes	ledger base URL
<code>INTERNAL_API_TOKEN</code>	both APIs	prod	guards <code>/api/internal/*</code>
<code>CORS_ORIGINS</code>	both APIs	prod	comma-separated; never <code>*</code> in prod
<code>VITE_LEDGER_API_URL</code>	web	yes	baked in at build time
<code>VITE_REPORTING_API_URL</code>	web	yes	baked in at build time

The two `VITE_*` values are **build-time**, not runtime. Change the API host? Rebuild the dashboard.

Health checks

Both APIs expose `GET /health` returning `{ status, service, version, uptimeSeconds, repository }`. Point the platform's health check at it and use it for zero-downtime rollout and rollback.

Docker

```
# APIs (build from the repo root)
docker build -f deployment/Dockerfile.ledger-api -t ledgerlab/ledger-api .
docker build -f deployment/Dockerfile.reporting-api -t ledgerlab/reporting-api .

# Dashboard (bake in the API URLs)
docker build -f deployment/Dockerfile.web \
  --build-arg VITE_LEDGER_API_URL=https://api.example.com \
  --build-arg VITE_REPORTING_API_URL=https://reports.example.com \
  -t ledgerlab/web .
```

What "production-scale" means here (G4)

- ☐ **Stateless** services — no local disk state; any replica can serve any request.
- ☐ **≥2 replicas** (or `min-instances ≥ 1 + max-instances` autoscaling).
- ☐ **Managed database** with backups and a private network path.
- ☐ **Connection pooling** sized to the plan; no connection-per-request.
- ☐ **Graceful shutdown**: `SIGTERM` drains in-flight requests and closes the pool (already wired in both `src/index.ts` files).
- ☐ **Health checks** gate the rollout; a failing replica is replaced.
- ☐ **Migrations as a separate step**, never on container boot.
- ☐ **Observability**: structured logs retained, an alert on 5xx and p95 latency.
- ☐ **Reproducible**: one documented command (or a committed blueprint/IaC) recreates the environment.

Rollback and migrations

- Keep migrations backward compatible for one release (add columns, don't rename/drop in the same deploy).
- Roll back the app by redeploying the previous image — the DB must still work with it.
- Never roll back by reverting a migration in production unless you have verified it is lossless.

Cost note

Everything above fits free/low tiers (Render starter, Cloud Run scale-to-zero, Fargate Spot, Container Apps consumption). Document what you actually used and what it costs.

7. Security

Deploying an accounting product means the data must be trustworthy and private. This checklist is what G5 is graded on. Each item needs evidence — a config, a command, or a screenshot.

1. Secrets

- ☐ No secret in git. `.env` is gitignored; only `.env.example` is committed.
- ☐ All secrets come from the platform store (Render env / AWS SSM / GCP Secret Manager / Azure Key Vault).
- ☐ `INTERNAL_API_TOKEN` is generated by the platform, not hard-coded.
- ☐ Database credentials use a dedicated, least-privilege user — not the managed master/root account.
- ☐ Rotate the token and DB password once and show the rotation works.

2. Service-to-service boundary

- ☐ `/api/internal/*` requires `Authorization: Bearer $INTERNAL_API_TOKEN` (implemented in `apps/ledger-api/src/routes/internal.ts`).
- ☐ The endpoint is **not** reachable from the public internet: - Render: keep it on the private network / block via a rule. - Cloudflare: a WAF rule blocks `POST /api/internal/*`. - AWS/GCP/Azure: restrict to the internal ALB / service network.
- ☐ Evidence: `curl -X POST https://<api>/api/internal/postings` returns `401 / 403 / 404` — never `200`.

3. CORS

- ☐ `CORS_ORIGINS` is the exact dashboard origin (`https://ledgerlab.example.com`).
- ☐ `*` is used **only** for local development.
- ☐ Preflight is verified: `curl -H "Origin: https://evil.example" -I ...` does not echo the origin back in `Access-Control-Allow-Origin`.

4. Transport and headers

- ☐ HTTPS only; HTTP redirects to HTTPS.
- ☐ HSTS (`max-age ≥ 31536000`, `includeSubDomains`, `preload`).
- ☐ `X-Content-Type-Options: nosniff`; `Referrer-Policy: strict-origin-when-cross-origin`; `X-Frame-Options: DENY` (or a CSP `frame-ancestors 'none'`).
- ☐ A Content-Security-Policy for the dashboard (see `deployment/nginx.conf`).
- ☐ TLS 1.2+ only; prefer Full (strict) at Cloudflare.

5. Input and output

- ☐ All request bodies validated with Zod at the boundary (routes use `parseBody`).
- ☐ Money parsed by `parseAmountToMinor`; floats rejected.
- ☐ API errors never leak stack traces, SQL, or secrets — `onError` maps `AppError` to a structured body and everything else to a generic 500.
- ☐ Pagination is bounded (`pageSize` $\leq$ 200).

6. Rate limiting and abuse

- ☐ A rate limit on `/api/*` (Cloudflare rule, Cloud Armor, WAF, or middleware).
- ☐ Document the chosen limit and the action (block/challenge).
- ☐ Evidence of a blocked burst.

7. Database

- ☐ Private network only; no public endpoint.
- ☐ Encryption at rest and in transit.
- ☐ Least-privilege app user (no DDL).
- ☐ Backups enabled and a restore tested.

8. Cloudflare layer (bonus, G6)

See `deployment/cloudflare/README.md`. Summary: proxied DNS, TLS Full (strict), WAF managed rules, rate limiting, `/api/internal` block, cache rules that bypass the API, and origin locking (Authenticated Origin Pulls or Cloudflare Tunnel).

9. Known gaps (state them honestly)

Real auth and multi-tenancy are **out of scope** for this test. In `docs/SUBMISSION.md`, describe how you *would* add them:

- Authentication: OIDC provider, short-lived tokens, refresh rotation.
- Authorisation: per-tenant row scoping; every query filtered by `tenant_id`.
- Audit: an append-only log of who posted/voided what and when.
- Isolation: separate schema or database per tenant for regulated customers.

Naming these gaps is better than pretending they do not exist.

8. AI usage policy

AI assistance is **allowed and expected**. This test is not about whether you use AI — it is about whether you can direct it, verify it, and explain your choices.

The one rule

If an AI helped produce something that ends up in the repo, that interaction is logged.

Use the provided script so the format is consistent and CI can verify it:

```
pnpm ai:log -- \
  --tool "opencode" \
  --model "deepseek/deepseek-v4.1-flash" \
  --subagent "ledger-architect" \
  --task "Fix Challenge A: trial balance asOf" \
  --prompt "buildTrialBalance ignores asOf. Make it filter postings by entryDate <= asOf." \
  --summary "Filtered postings before aggregation; kept balanced flag computed from filtered rows" \
  --decision "accepted" \
  --files "packages/shared/src/reporting.ts"
```

- Writes docs/ai/prompt-log.jsonl (source of truth) and regenerates docs/AI-PROMPT-LOG.md (human-readable table).
- Read a long prompt from stdin: pbpaste | pnpm ai:log -- --tool ... --prompt -
- pnpm ai:verify fails CI on missing or incomplete records.

Required fields

FIELD	MEANING
tool / model	What produced the output
subagent	Which sub-agent (or blank)
task	The goal of the interaction
prompt	What you actually asked (verbatim, trimmed is fine)
summary	What changed as a result
decision	accepted · edited · rejected · reverted
files	Files the change touched

What is scored

- **Completeness.** Every non-trivial change traces to a logged interaction.

- **Honesty.** `decision: rejected` entries are *valuable* — they show judgment. A log with only sparkling successes reads as fiction.
- **Verification.** You confirm the AI's output (tests, types, manual check) before trusting it. Say so in `summary`.
- **Direction.** Your prompts get sharper over time: context, constraints, acceptance criteria.

What is penalised

- Editing the challenge specs so they pass without fixing the source.
- Committing secrets an AI suggested.
- A prompt log that is empty, fabricated, or back-filled in one commit at the end.
- Shipping unverified AI output — e.g. floats for money, or a `CORS_ORIGINS=*` in production.

Suggested workflow

1. Ask the relevant sub-agent for a **plan** before code (log it).
2. Apply the change yourself or let the agent edit; keep the diff small.
3. Run `pnpm typecheck && pnpm test`. If it fails, that is a **new** interaction.
4. Log the decision: accepted / edited / rejected, and why.
5. Commit the code together with its log entries — one story per commit.

Privacy

Do not paste real customer data, credentials, or proprietary third-party source into any AI tool. Use the demo data in `packages/shared/src/seed.ts`.

9. Sub-agents

You are required to **build your own sub-agents** and use them for the work (G8). Four starter agents are provided in `.opencode/agent/`; extend or replace them. An agent you actually used and can demonstrate scores; a folder of unused boilerplate does not.

What a sub-agent is

A narrowly-scoped collaborator with its own system prompt, tools, and definition of done. Good sub-agents:

- own **one** concern (the ledger invariant, the design rules, the deploy);
- are **read-only** when they review and **write-enabled** only when they build;
- produce **evidence**, not opinions;
- have an explicit output contract so their results are checkable.

Provided starters

AGENT	MODE	JOB
<code>ledger-architect</code>	read-only	Guards the double-entry invariant and posting rules
<code>ui-unslop</code>	write	Enforces <code>docs/DESIGN.md</code> on every screen
<code>test-runner</code>	read-only	Runs typecheck/test/build, reports the minimal fix
<code>deploy-security</code>	write	Production-scale + security review of the deploy

What you must do

- ☐ Keep **at least three** working agents (they may be the starters, improved).
- ☐ Add **at least one** agent of your own that fits your workflow. Likely candidates:

SUGGESTED AGENT	JOB
<code>db-migrator</code>	Schema, migrations, seeding, repository port; refuses <code>REAL</code> money columns
<code>reporting-verifier</code>	Independently recomputes reports and proves they tie out
<code>challenge-fixer</code>	Works only on <code>src/challenges/*</code> ; returns source diffs, never test edits
<code>api-contract</code>	Keeps the API surface, Zod schemas, and OpenAPI in sync
<code>perf-probe</code>	Load-tests an endpoint and reports p50/p95 before and after

- ☐ Write, for each agent, the exact prompt you used and the outcome — logged via `pnpm ai:log` with `--subagent <name>`.
- ☐ Demonstrate at least one agent **catching a real defect** and the fix that followed.

Format

An agent is a markdown file in `.opencode/agent/` with YAML frontmatter:

```
---
description: When to use this agent, in one sentence.
mode: subagent
temperature: 0.1
tools:
  write: false
  edit: false
  bash: true
---

The system prompt: scope, method, constraints, output contract.
```

Definition of done for each agent

1. **Scoped** — one concern, stated in the first line of the prompt.
2. **Tooled** — read-only unless it must write; no unnecessary `bash`.
3. **Evidence-based** — every claim ties to a file, command, or test output.
4. **Contract** — a fixed output shape a human can verify at a glance.
5. **Used** — at least one logged interaction referencing it.
6. **Bounded** — it says what it will *not* do (scope creep is a defect).

Anti-patterns

- A "do everything" agent that owns the whole repo.
- An agent that claims success without pasting command output.
- An agent that edits tests to make them pass.
- Agents that duplicate each other's scope.

10. Infrastructure plan (template)

Deliverable. Fill this in and commit it as `docs/INFRASTRUCTURE-PLAN.md`. It is scored (G9) and the bank's reviewer will read it before they approve the embed. This file ships as a template — replace every `TODO` and delete the guidance you do not need.

Author	TODO
Date	TODO
Target environment	TODO (Render / AWS / GCP / Azure)
Status	Draft / Review / Approved
Related	<code>SECURITY.md</code> , <code>DEPLOYMENT.md</code> , <code>DATABASE.md</code>

1. Executive summary

TODO — five bullets a non-engineer can read. State the target architecture, the durability guarantee, the cost, and what you deliberately did not build.

- Compute: ...
- Data: ...
- Edge: ...
- Reliability target: ... (availability, RPO, RTO)
- Cost: ... per month at current traffic

2. Current vs target

CONCERN	TODAY (VIBE-CODED)	TARGET	WHY IT MATTERS
Compute	One VM, one process	TODO	Survive payday traffic
Data	SQLite file, no backups	TODO	No total data loss
Networking	Public DB, * CORS	TODO	Bank review
Secrets	Shared password in repo	TODO	Credential compromise
Observability	None	TODO	Prove 99.9%
Deploy	Manual, on the box	TODO	Reproducible, rollback-able

3. Architecture

TODO — one diagram (Mermaid or ASCII) of the target. Show: DNS/edge → load balancer → services → database, plus secret store, registry, and observability. Label the trust boundaries.

```
flowchart LR
    U[Users] --> CF[Cloudflare WAF/CDN]
    CF --> LB[Load balancer]
    LB --> W[web: static SPA]
    LB --> A[ledger-api]
    LB --> R[reporting-api]
    A --> DB[(PostgreSQL)]
    R --> A
```

4. Environments

ENVIRONMENT	PURPOSE	DATA	ACCESS	NOTES
Local	Developer	In-memory or local PG	Any	Zero-config default
Staging	Pre-prod, migrations rehearsal	Anonymised subset	Team	Same shape as prod
Production	Customers	Real	Break-glass only	Private networking

TODO — say how config differs per environment and how secrets are scoped. Do not share credentials across environments.

5. Compute, scaling and capacity

- Service sizing (CPU/memory per service) and **why** (show the math in §13).
- Autoscaling policy: min/max replicas, the metric it scales on, and cooldowns.
- Concurrency and timeouts per service.
- Cold-start strategy (keep the ledger service warm; `min-instances ≥ 1`).
- Graceful shutdown: in-flight drain on SIGTERM (already wired — verify it).

SERVICE	VCPU	MEMORY	MIN	MAX	SCALE METRIC	TIMEOUT
ledger-api	TODO	TODO	TODO	TODO	TODO	TODO
reporting-api	TODO	TODO	TODO	TODO	TODO	TODO

6. Data and durability

- Engine, version, and why (see `DATABASE.md`).
- HA / failover posture.
- **Backups:** frequency, retention, encryption, cross-region?

- **Restore drill:** date performed, duration, result. An untested backup is not a backup — asked for explicitly by the bank.
- Migration strategy: forward-only, one release of backward compatibility, applied as a deploy step (not on boot).
- Connection pooling: pool size, max idle, and how it maps to the DB plan.
- Data retention and PII minimisation.

GUARANTEE	VALUE	HOW IT IS MET
Recovery Point Objective (RPO)	TODO	TODO
Recovery Time Objective (RTO)	TODO	TODO
Backup frequency / retention	TODO	TODO
Restore drill result	TODO	TODO

7. Networking, DNS, TLS and edge

- DNS and domain (a real TLD, per G6).
- TLS termination points; certificate management and renewal.
- WAF, rate limiting, and bot controls (see `deployment/cloudflare/`).
- Private networking between services and DB; which ports are exposed.
- Egress rules and outbound dependencies.

8. Secrets and identity

- Secret store and how each secret is injected (never baked into an image).
- Service identities / roles and their least-privilege policies.
- Rotation policy for `INTERNAL_API_TOKEN` and DB credentials.
- Who can read production secrets, and how that access is audited.

9. CI/CD and release

STAGE	TRIGGER	WHAT RUNS	GATE
PR	push	format, typecheck, test, ai:verify	Required
Build	merge to main	image build, SBOM, tag	—
Deploy staging	tag	migrate, deploy, smoke	Manual
Deploy prod	approval	migrate, canary/rolling, health check	Manual
Rollback	on-call	redeploy previous image	—

TODO — state the rollback procedure and the migration-compatibility rule.

10. Observability and SLOs

- Logs: where, retention, structured fields.
- Metrics: the four golden signals per service.
- Traces: sampled? what is instrumented?
- SLOs: availability, p95 latency, error rate — with the alert thresholds and the **error budget** policy (what happens when it is spent).

SLO	TARGET	ALERT IF	OWNER
Availability	99.9%	5xx > 1% for 5 min	TODO
Latency (ledger reads)	p95 < 300 ms	p95 > 300 ms for 10 min	TODO
Freshness (reports)	< 60 s stale	TODO	TODO

11. Security controls

Map each control to the checklist in `SECURITY.md`. Do not duplicate it — reference it and record the evidence location.

CONTROL	IMPLEMENTED	EVIDENCE
No secrets in git	TODO	TODO
<code>/api/internal/*</code> blocked at edge	TODO	TODO
CORS restricted	TODO	TODO
HSTS + secure headers	TODO	TODO
Rate limiting on <code>/api/*</code>	TODO	TODO
DB least-privilege user	TODO	TODO

12. Cost model

Estimate monthly cost at **current traffic (1x)**, **3x** (the next phase), and **10x** (where the model breaks). Name the line items and the plan you chose.

LINE ITEM	1x	3x	10x	NOTES
Compute (services)	TODO	TODO	TODO	
Database	TODO	TODO	TODO	
Edge / CDN / WAF	TODO	TODO	TODO	
Backups / storage	TODO	TODO	TODO	
Observability	TODO	TODO	TODO	
Total / month	TODO	TODO	TODO	

Also state: the **first thing you would spend money on** as traffic grows, and the metric that triggers it.

13. Failure modes and DR runbook

FAILURE	BLAST RADIUS	DETECTION	RESPONSE	TESTED?
Database unavailable	All writes fail	Health/alerts	TODO	TODO
Ledger service crash-loop	No posting	Health check	TODO	TODO
Bad migration	Wrong data	Canary + checks	TODO	TODO
Region outage	Everything	Status page	TODO	TODO
Secret rotation failure	Auth broken	401 spike	TODO	TODO

Include the capacity math you used (requests/sec per replica, DB connections needed, storage growth/month). Show the arithmetic.

14. Decision log (ADRs)

For each significant choice: **context** → **options considered** → **decision** → **why** → **consequences**. Minimum three. Example:

ADR-001 — Managed PostgreSQL over self-hosted. Considered self-managed Postgres on a VM, RDS/Cloud SQL/Flexible Server, and staying on SQLite. Chose managed because tested PITR backups and failover are requirements, and operating a database is not the client's differentiator. Consequence: higher unit cost, less control over extensions.

How this is scored (G9)

- ☐ Every section is filled in with specifics, not adjectives.
- ☐ The diagram matches what is actually deployed (test it against reality).
- ☐ RPO/RTO are stated **and** a restore was actually performed.
- ☐ The cost model shows arithmetic and names the 10× breaking point.
- ☐ At least three ADRs with genuine rejected options.
- ☐ Controls reference `SECURITY.md` with evidence paths.
- ☐ A reviewer could rebuild the environment from this document alone.

11. Next-phase development plan (template)

Deliverable. Fill this in and commit it as `docs/NEXT-PHASE-PLAN.md`. It is scored (G10). The seed investors will read it during due diligence, and Kira will use it to decide what to fund next. Replace every `TODO`.

Author	TODO
Date	TODO
Horizon	Next phase (e.g. 90 days)
Phase	Phase 2 — from "trustworthy" to "growing"
Related	<code>INFRASTRUCTURE-PLAN.md</code> , <code>CLIENT-STORY.md</code>

1. Where we are

TODO — one short paragraph on the state **after** this engagement: correct ledger, real database, deployed and monitored, security review passing, X paying customers, current constraints.

2. Outcomes for this phase

Define 3–5 outcomes, not a feature list. Each has a **measurable** success signal and the stakeholder who cares.

#	OUTCOME	SUCCESS SIGNAL	FOR WHOM
O1	Bank embed live for first cohort	100 warungs approved for working capital	Bank / Kira
O2	Books survive an audit	CPA sign-off, zero unbalanced entries	CPA
O3	TODO	TODO	TODO
O4	TODO	TODO	TODO

Candidate outcomes to consider (pick, don't sprawl): authentication and multi-tenancy, an append-only audit trail, multi-currency, bank API integration, mobile capture, statement import, accountant collaboration.

3. Prioritisation

Score the candidate work. Use RICE (Reach × Impact × Confidence ÷ Effort) or state your own method. The **method** matters more than the exact numbers.

INITIATIVE	REACH	IMPACT	CONFIDENCE	EFFORT	SCORE	DECISION
Auth + tenant scoping	TODO	TODO	TODO	TODO	TODO	Now
Audit trail	TODO	TODO	TODO	TODO	TODO	Now
Multi-currency	TODO	TODO	TODO	TODO	TODO	Next
Bank ledger export API	TODO	TODO	TODO	TODO	TODO	Now
TODO	TODO	TODO	TODO	TODO	TODO	Later

TODO — note what you are explicitly **not** doing and why. Saying no is part of the plan.

4. Milestones

Three milestones, each with **exit criteria** that are testable, and a date. Work backward from the bank go-live.

MILESTONE	WINDOW	CONTENTS	EXIT CRITERIA
M1 — Harden	Weeks 1–4	TODO	TODO
M2 — Embed	Weeks 5–8	TODO	TODO
M3 — Scale	Weeks 9–13	TODO	TODO

5. Delivery plan

- **Capacity:** who does the work (you? a new hire?) and how much of their time.
- **Team shape:** the next two hires and why, in priority order.
- **Dependencies:** external teams (the bank, the CPA), and their lead times.
- **Cadence:** release rhythm, demo schedule, stakeholder updates.
- **Definition of done** for a work item on this team.

ROLE	NEEDED BY	WHY	COST SIGNAL
TODO	TODO	TODO	TODO

6. Technical workstreams

Map each workstream to the infrastructure plan and the current codebase. Name the files/services that will change.

WORKSTREAM	DEPENDS ON	FIRST PR	RISK
Authentication + tenancy	—	TODO	TODO
Append-only audit log	DB schema	TODO	TODO
Multi-currency reporting	packages/shared/reporting	TODO	TODO

WORKSTREAM	DEPENDS ON	FIRST PR	RISK
Bank export API	Infra plan §7	TODO	TODO

7. Risks and mitigations

RISK	LIKELIHOOD	IMPACT	MITIGATION	OWNER
Bank review slips	TODO	High	TODO	TODO
Schema migration on live money data	TODO	High	TODO	TODO
Single engineer bus factor	TODO	High	TODO	TODO

8. Metrics

The numbers this phase must move, with current values and targets.

METRIC	NOW	TARGET	SOURCE
Weekly active businesses	1,400	TODO	TODO
% entries posted without support contact	TODO	TODO	TODO
p95 ledger latency	TODO	< 300 ms	Monitoring
Availability	TODO	99.9%	Monitoring
Support tickets / 100 businesses	TODO	TODO	Helpdesk

9. Explicitly deferred

ITEM	WHY DEFERRED	REVISIT WHEN
Spanish/English localisation	TODO	TODO
Native mobile app	TODO	TODO
Real-time collaborative editing	TODO	TODO

10. Decision log

At least two ADRs for phase-shaping decisions (e.g. "build auth vs buy an identity provider"). Context → options → decision → consequences.

How this is scored (G10)

- ☐ Outcomes are measurable and tied to a named stakeholder.
- ☐ Prioritisation uses a stated method with real trade-offs, not vibes.
- ☐ Milestones have testable exit criteria and dates that work backward from the bank go-live.

- ☐ The plan is honest about capacity (one engineer) and names the next hires.
- ☐ It says no to things, with reasons.
- ☐ Metrics have a source of truth, not guesses.
- ☐ It is a plan to *learn*, not a wish list of features.

12. Submission template

Fill this in. Replace every `TODO` . Reviewers read this first.

Candidate: `TODO` **Date:** `TODO` **Time spent:** `TODO`

Deployed URLs

SURFACE	URL	/HEALTH
Dashboard	<code>TODO</code>	—
Ledger API	<code>TODO</code>	<code>TODO</code>
Reporting API	<code>TODO</code>	<code>TODO</code>

Cloudflare / custom domain: `TODO` (or "not attempted")

What I changed

`TODO` — bullet the meaningful changes, grouped by goal (G0...G8). Link commits or files.

Challenge suite

```
pnpm --filter @ledgerlab/ledger-api test:challenges
# TODO paste the (green) output
```

- Challenge A (trial balance `as0f`): `TODO` how you fixed it and why.
- Challenge B (void transition): `TODO` how you fixed it and why.

Database

- Engine and version: `TODO`
- Migrations: `TODO` (command)
- Seeding: `TODO` (command)
- Why this engine: `TODO`

Deployment

- Target: `TODO` (Render / AWS / GCP / Azure)
- Reproduce it: `TODO` (commands or blueprint path)
- Scaling: `TODO` (replicas / min-max instances)
- Secrets: `TODO` (where they live)
- Migrations as a deploy step: `TODO`

Security

- `INTERNAL_API_TOKEN` evidence: `curl -X POST https://.../api/internal/postings` → TODO status
- CORS: TODO (origin, and the preflight check result)
- Headers/HSTS: TODO (`curl -I` output)
- Rate limiting: TODO (limit + action)
- Known gaps: auth/multi-tenancy not built — TODO how you would add them

Cloudflare + TLD (bonus)

- Domain: TODO
- `dig +short` output: TODO
- TLS mode: TODO (Full strict) + screenshot/commit
- WAF rule and rate-limit rule: TODO (paste the expression)
- `/api/internal/*` blocked at edge: TODO (evidence)
- Cache rules: TODO

Infrastructure plan (G9)

- Document: `docs/INFRASTRUCTURE-PLAN.md` — TODO (status: complete / partial)
- Topology summary in one sentence: TODO
- RPO / RTO: TODO
- Restore drill: date TODO, duration TODO, result TODO
- Cost at 1x / 3x / 10x: TODO
- First bottleneck at 10x: TODO
- ADRs: TODO (list the decisions)

Next-phase plan (G10)

- Document: `docs/NEXT-PHASE-PLAN.md` — TODO (status: complete / partial)
- Outcomes: TODO (list the 3–5)
- Prioritisation method and top initiative: TODO
- Milestones + exit criteria: TODO
- Next hires: TODO
- Explicitly deferred: TODO

AI usage

- Entries in `docs/ai/prompt-log.jsonl`: TODO (count)
- A prompt I **rejected** and why: TODO
- How I verified AI output: TODO

Sub-agents

AGENT	FILE	WHAT IT DID
TODO	<code>.opencode/agent/...</code>	TODO

- Defect a sub-agent caught: TODO

Verification

```
pnpm format:check    # TODO PASS/FAIL
pnpm typecheck       # TODO
pnpm test            # TODO (n passed)
pnpm build           # TODO
pnpm ai:verify       # TODO
```

What I skipped and why

TODO — be explicit. Underscoping is fine; hiding it is not.

If I had more time

TODO — ranked, with the first thing you would do next.